



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_Compiler Design

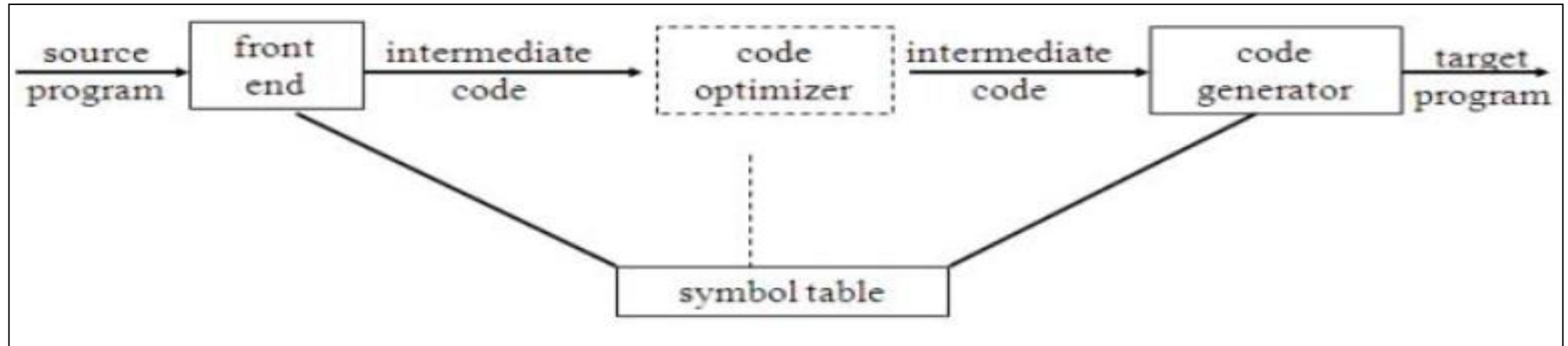


Code Generation

**Dr. B.V. Baiju,
SCOPE,
Assistant Professor
VIT, Vellore**

CODE GENERATION

- A code generator is a **machine-dependent phase** of the compiler because whatever the input is given to the code generator, the code generator converts the input into the final target code.
- It uses **registers** to store the operands of the three address statement.
- **Code generator** converts the intermediate representation of source code into a form that can be readily executed by the machine.
- Designing of the code generator should be done in such a way that it can be easily implemented, tested, and maintained.



Generating Code for Assignment Statements

- The assignment statement **d:= (a-b) + (a-c) + (a-c)** can be translated into the following sequence of three address code:

t:= a-b
u:= a-c
v:= t +u
d:= v+u

Code Sequence

Statement	Code Generated	Register Descriptor Register Empty	Address descriptor
t:= a - b	MOV a, R0 SUB b, R0	R0 contains t	t in R0
u:= a - c	MOV a, R1 SUB c, R1	R0 contains t R1 contains u	t in R0 u in R1
v:= t + u	ADD R1, R0	R0 contains v R1 contains u	u in R1 v in R1
d:= v + u	ADD R1, R0 MOV R0, d	R0 contains d	d in R0 d in R0 and memory

Issues in the Design of a Code Generator

- The most important criterion for a code generator is that it produce correct code.
- The following issue arises during the code generation phase:

Input to the Code Generator

The Target Program

Instruction Selection

Register Allocation

Evaluation Order

Input to the Code Generator

- The input to the code generator contains the *intermediate representation* of the source program and the *information of the symbol table*.
 - The source program is produced by the front end.
 - The many choices for the IR include
 - **Three-address representations** such as
 - **Quadruples**
 - **Triples**
 - **Indirect triples**
 - **Virtual machine representations** such as
 - **Bytecodes**
 - **Stack-machine code**
 - **Linear representations** such as
 - **Postfix notation**
 - **Graphical representations** such as
 - **Syntax trees**
 - **DAG's**
- Input to code generation is assumed to be error-free.

The Target Program

- The **Instruction-set architecture** of the target machine has a significant impact on the difficulty of constructing a good code generator that produces high-quality machine code.
- The most common **target-machine architectures** are
 - **RISC (Reduced Instruction Set Computer)**
 - **CISC (Complex Instruction Set Computer)**
 - **Stack Based**

RISC (Reduced Instruction Set Computer)

- A RISC machine typically has many
 - **Registers**
 - **Three-address instructions**
 - **Simple addressing modes**
 - **A relatively simple instruction-set architecture.**

ADD R1, A, B	; R1 $\leftarrow$ M[A] + M[B]
ADD R2, C, D	; R2 $\leftarrow$ M[C] + M[D]
MUL Y, R1, R2	; M[Y] $\leftarrow$ R1 * R2

CISC (Complex Instruction Set Computer)

- A CISC machine typically has
 - *Few registers*
 - *Two-address instructions*
 - *A variety of addressing modes*
 - *Several register classes*
 - *Variable-length instructions*
 - *Instructions with side effects*

CPU REGISTERS	
REGISTER	SYMBOL
Program Counter	PC
Instruction Register	IR
Accumulator Register	AC
Memory Buffer Register	MBR
Memory Address Register	MAR

Using two address instructions

MOV R1, A	; R1 $\leftarrow$ M[A]
ADD R1, B	; R1 $\leftarrow$ R1 + M[B] (Perform A+B)
MOV R2, C	; R2 $\leftarrow$ M[C]
ADD R2, D	; R2 $\leftarrow$ R2 + M[D] (Perform C + D)
MUL R1, R2	; R1 $\leftarrow$ R1 * R2 (Perform (A+B)*(C+D))
MOV Y, R1	; M[Y] $\leftarrow$ R1

Implied Mode

Immediate Mode

Register Mode

Register Indirect Mode

Autodecrement Mode

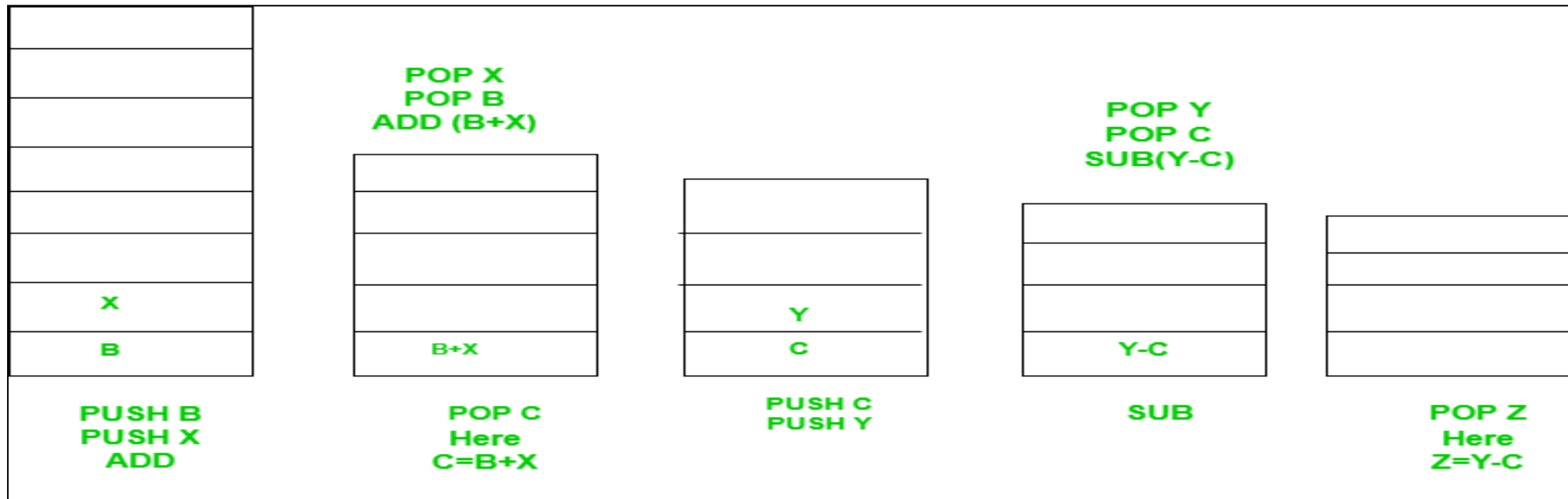
Autoincrement Mode

Direct Address Mode

Indirect Address Mode

Stack based

- Operations are done by ***pushing operands onto a stack*** and then performing the **operations on the operands at the top of the stack**.
- To achieve high performance the top of the stack is typically kept in registers.
- Stack-based machines almost disappeared because it was felt that the stack organization was too limiting and required too many swap and copy operations.



- Stack-based architectures were revived with the introduction of the **Java Virtual Machine (JVM)**.

- The target program is the output of the code generator.
- The output may be
 - **Absolute Machine Language**
 - It can be placed in a fixed location in memory and can be executed immediately.
 - Can be used for small programs
 - **Relocatable Machine Language (*object module*)**
 - Allows subprograms to be compiled separately.
 - A set of relocatable object modules can be linked together and loaded for execution by a linking loader.
 - **Assembly Language**
 - It makes the process of code generation easier.

Instruction Selection

- The code generator must **map the IR program into a code sequence** that can be **executed by the target machine**.
- The complexity of performing this mapping is determined by a factors such as
 - *the level of the IR*
 - *the nature of the instruction-set architecture*
 - *the desired quality of the generated code*
- If the IR is high level, the code generator may translate each IR statement into a sequence of machine instructions using code templates.
- The nature of the instruction set of the target machine has a strong effect on the difficulty of instruction selection.
- The uniformity and completeness of the instruction set are important factors.
- Floating-point operations are done using separate registers.
- **Instruction speeds** and **machine idioms** are important factors when efficiency of target program is considered.
- The quality of the generated code is determined by its speed and size.

- Every three-address statement of the form $x = y + z$, where x , y , and z are statically allocated, can be translated into the code sequence

LD RO, y	// RO = y	(load y into register RO)
ADD RO, RO, z	// RO = RO + z	(add z to RO)
ST x, RO	// x = RO	(store RO into x)

- This strategy often produces redundant loads and stores.

a := b + c	MOV b, R0	b → R0		MOV b, R0
d := a + e	ADD c, R0	R0 → c + R0		ADD c, R0
	MOV R0, a	R0 → a	→	ADD e, R0
	MOV a, R0	a → R0		MOV R0, d
	ADD e, R0	R0 → e + R0		
	MOV R0, d	R0 → d		

- If the target machine has an "increment" instruction (INC), then the three-address statement $a = a + 1$ may be implemented more efficiently by the single instruction INC a.

a = a + 1		
MOV a, R0		INC a
ADD #1, R0	→	
MOV R0, a	This can be done with one instruction	It is a register operator performs faster

Register Allocation

- Register can be accessed faster than memory.
- The instructions involving operands in register are shorter and faster than those involving in memory operand.
- The use of registers is often subdivided into two subproblems

Register Allocation

- We select the set of variables that will reside in the register.
- Which register contains which variable
- $R0 \leftarrow a, R1 \leftarrow b$

Register Assignment

- We pick the register that contains a variable.
- Which variable contains which register.
- $a \rightarrow R0, b \rightarrow R1$

Register Allocation

R1<--- a
R2<--- b
R3<--- c
R4<--- d



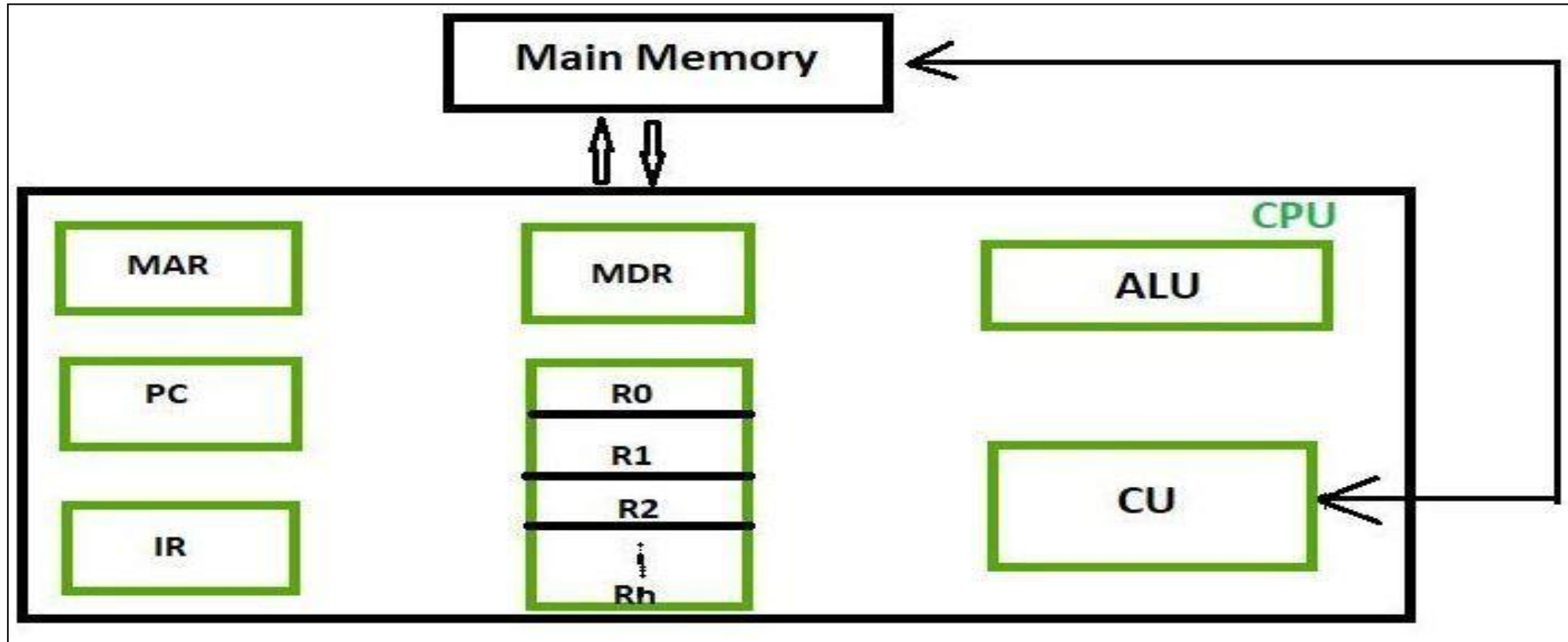
MOV R3, c
MOV R4, d
MUL R3, R4
MOV R2, b
ADD R2, R3
MOV R1, R2
MOV a, R1

Evaluation Order

- The code generator decides the order in which the instruction will be executed.
- The order of computations affects the efficiency of the target code
- Many computational orders will only require a few registers to store interim results.
- Picking up the best order is one difficulty in code generation.

Target Machine

- A target machine is a **byte-addressable machine** with 4 bytes for a word.
- The target machine has a **general-purpose register** $R_0, R_1, \dots, R_{n-1}$.



- The component of instruction is an operator, followed by a target, and then followed by a list of source operands.

- The target machine also has three address instructions

op, source, destination

- **op** is used as an op-code
 - **source , destination** are used as a data field.
- The target machine has the following op-code:
 - **ADD** (it adds source to destination)
 - **SUB** (it subtracts source from destination)
 - **MOV** (it moves source to destination)
- The source and destination of an instruction are specified by combining registers and memory locations with address modes.

Instructions available in the target machine are as follows:

(i) Load operations (load data from memory (directly or indirectly) into a register)

- The instruction

LD *dst, addr* //loads the value in location *addr* into location *dst*.

- This instruction denotes the assignment $dst = addr$.
- Most common form of this instruction

LD *r, x* // loads the value in location *x* into register *r*

LD *R0, R1* //copies the contents of register *R1* into register *R0*

LD *R0, M* //Loads contents of memory location *M* into register *R0*

(ii) Store operations

- The instruction

ST *r, x* //Instruction stores the value in location *x* to register *r*

iii) Computation Operations

- Instruction format

opcode	address	address	address
--------	---------	---------	---------

OP, dst, src1, src2

OP are the add or sub operator **dst**, **src1**, and **src2** are locations

- Instructions are in form

ADD dst, src1, src2

- The locations may or may not be distinct.

SUB n, r₂, r₃

Computes $n = r_2 - r_3$

iv) Unconditional jump

- The instruction

BR L

- This operation jumps from branch BR to level L

v) Conditional jumps

- The general form of this operation is

Bcond, r, L

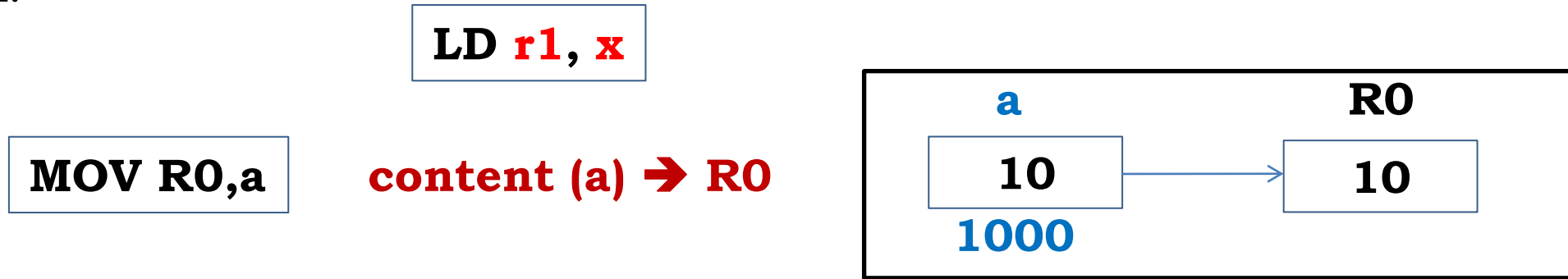
r is a register

L is a label

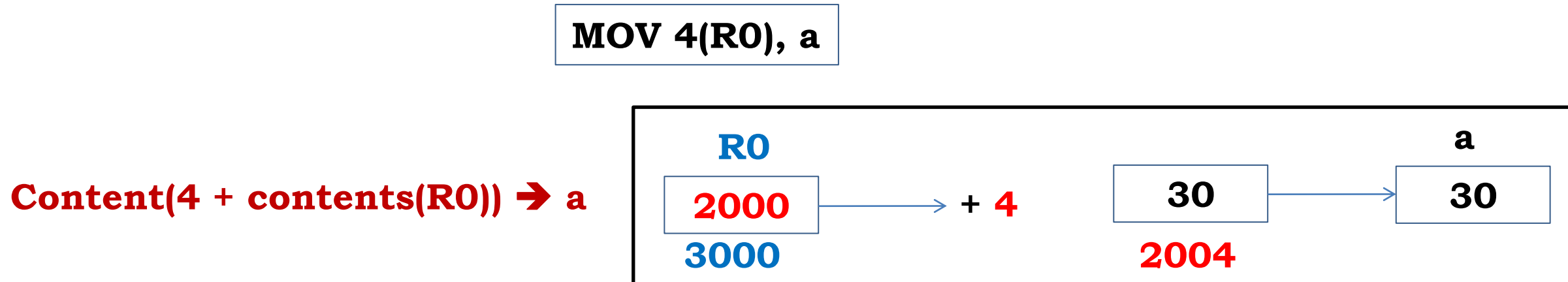
cond stands for any of the common tests on values in the register r.

Target machine has a variety of **addressing modes**

- In instruction, a **variable name** x means there is a location in memory reserved for x .



- A location can also be an **indexed address** of the form $a(r)$, where a is a variable and r is a register.
- The memory location denoted by $a(r)$ is computed by taking the i-value of a and adding to it the value in register r .

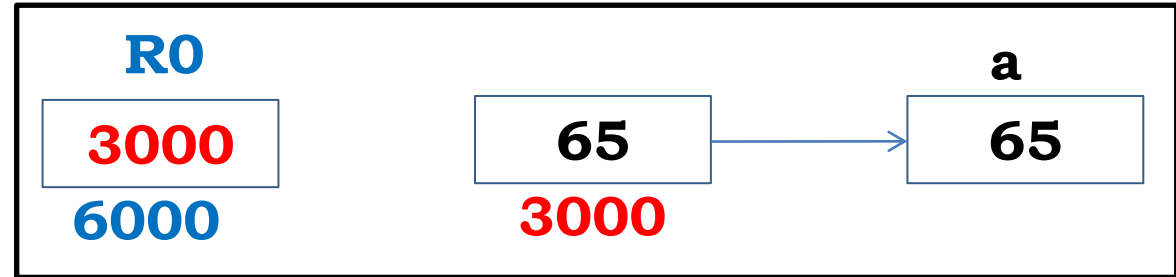


- There are two **indirect addressing modes**:
 - Indirect register**: The memory location found in the location represented by the contents of register r

***r**

MOV *R0, a

Content(contents(R0)) → a

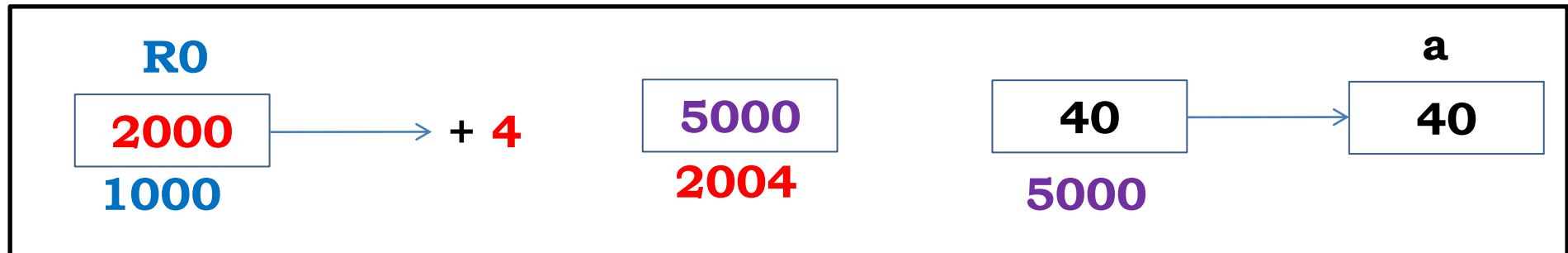


- Indirect indexed**: The memory location found in the location obtained by adding **C(100)** to the contents of r

***C(r)**

MOV *4(R0),a

Content(contents(4 + contents(R0))) → a



- **Immediate constant addressing mode.**
- The constant is prefixed by #.

MOV #4,R0

R0

4

MODE	FORM	ADDRESS	ADDED COST
<i>Variable</i>	X	X	1
<i>Register</i>	R	R	0
<i>Indexed</i>	c(R)	C+ contents(R)	1
<i>Indirect register</i>	*R	contents(R)	0
<i>Indirect indexed</i>	*c(R)	contents(c+ contents(R))	1
<i>Immediate constant or Literal</i>	#c	c	1

- The three-address statement $\mathbf{x} = \mathbf{y} - \mathbf{z}$ can be implemented by the machine instructions

LD R1, y	// R1 = y
LD R2, z	// R2 = z
SUB R1, R1, R2	// R1 = R1 - R2
ST x, R1	// x = R1

- One of the goals of a good code-generation algorithm is to avoid using all four of these instructions, whenever possible.

LD R1, y
SUB R1, R1, z
ST x, R1

- Suppose `a` is an array whose elements are **8-byte values**, perhaps real numbers.
- Also assume elements of `a` are indexed starting at 0.
- We may execute the three-address instruction **`b = a [ i ]`** by the machine instructions

```
LD R1, i
MUL R1, R1, 8
LD R2, a(R1)
ST b, R2
```

```
// R1 = i, Load the value of i to R1
// R1 = R1 * 8
// R2 = contents(a + contents(R1))
// b = R2
```

- The assignment into the array `a` represented by three-address instruction **`a [ j ] = c`** is implemented by

```
LD R1, c
LD R2, j
MUL R2, R2, 8
ST a(R2), R1
```

```
// R1 = c, Load c to R1
// R2 = j
// R2 = R2 * 8
// contents(a + contents(R2)) = R1
```

- To implement a **simple pointer indirection**, such as the three-address statement **x = *p**, we can use machine instructions

LD R1, p LD R2, 0 (R1) ST x, R2	// R1 = p // R2 = contents(0 + contents(R1)) // x = R2
--	--

- The **assignment through a pointer *p = y** is similarly implemented in machine code by

LD R1,p LD R2, y ST 0 (R1), R2	// R1 = p // R2 = y // contents(0 + contents(R1))= R2
---	---

- Consider a **conditional-jump** three-address instruction like **if x < y goto L**

LD R1, x LD R2, y SUB R1, R1, R2 BLTZ R1, M	// R1 = x // R2 = y // R1 = R1 - R2 // if R1 < 0 jump to M	BLTZ – Branch Less than Zero
--	---	-------------------------------------

M is the label that represents the first machine instruction generated from the three-address instruction that has label L.

Instruction Costs

- Common cost measures are
 - *Length of compilation time and size*
 - *Running time*
 - *Power consumption*
- Each target language instruction has an associated cost
- Instruction cost is

1 + cost of source and destination addressing modes
- Addressing modes involving **registers** have **zero** additional cost, while those involving a **memory location** or **constant** in them have an additional **cost of one**

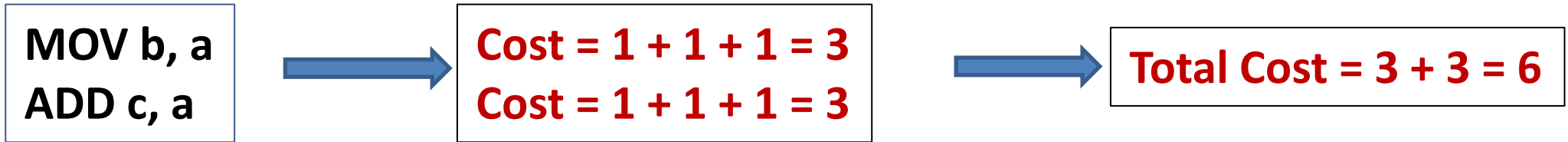
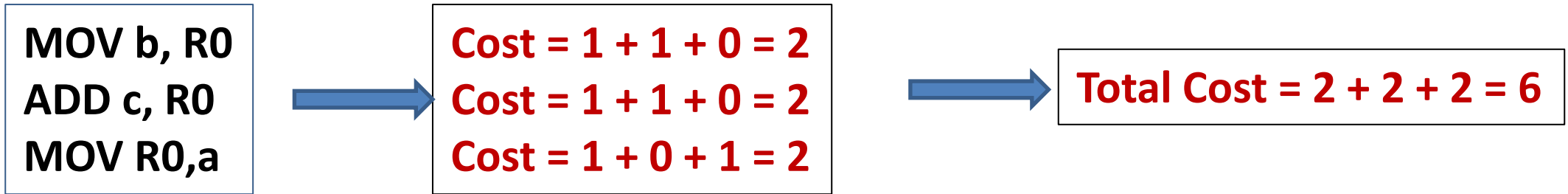
Instruction operation	Cost	
MOV R0, R1 (Register to register)	1	1+0+0
MOV R0, M (Register to memory)	2	1+0+1
MOV M, R0 (Memory to register)	2	1+1+0
MOV 4(R0), M (4+contents(R0) into memory)	3	1+1+1
ADD #1, R0	2	1+1+0

Instruction Operation	cost	
ADD #1, R0	3	1+1+0
ADD 4(R0),*12(R1)	3	1+1+1
LD R0,R1	1	1+0+0
LD R0,M	2	1+0+1
LD R1, *100(R2)	2	1+0+1
SUB 2(R0), *8(R1)	3	1+1+1

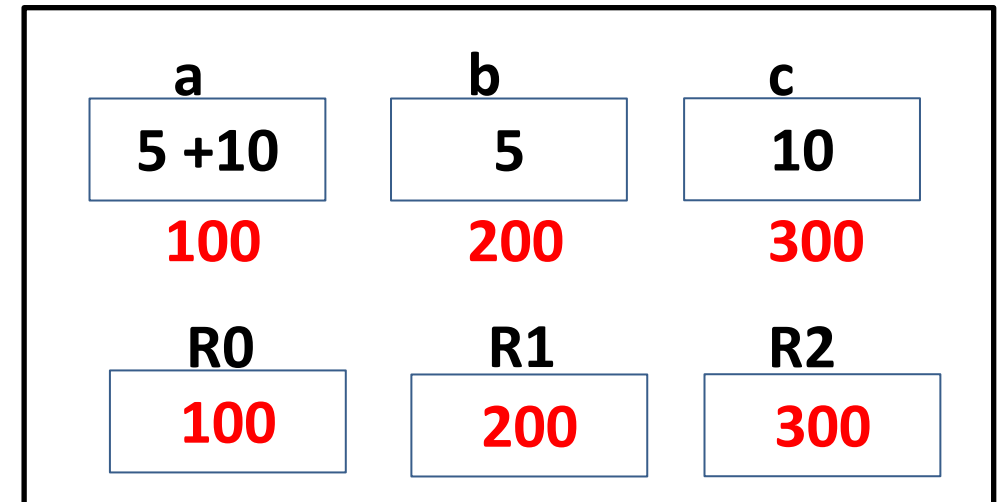
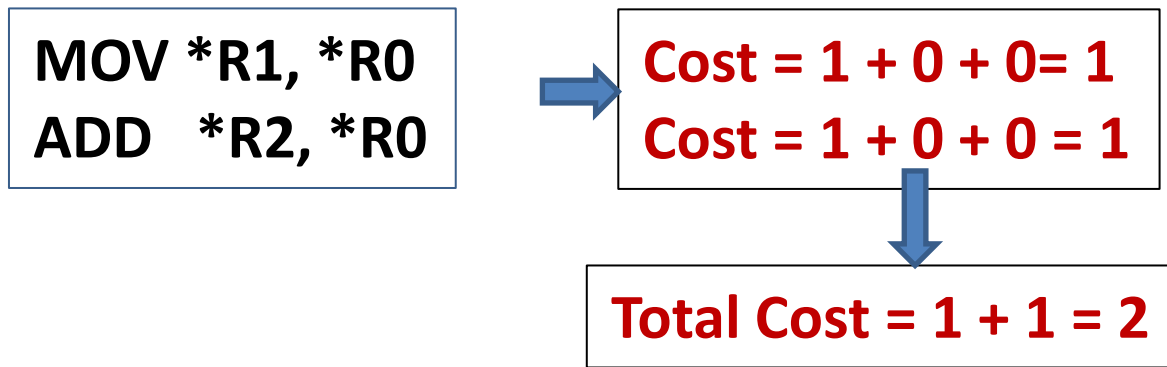
MODE	FORM	COST
<i>Variable</i>	X	1
<i>Register</i>	R	0
<i>Indexed</i>	c(R)	1
<i>Indirect register</i>	*R	0
<i>Indirect indexed</i>	*c(R)	1
<i>Immediate constant</i>	#c	1

- Example: Consider the three address statement **a = b + c**

a, b, c are distinct memory location



Assume R0,R1,R2 contains address of a,b,c (Use Indirect Register)



- Determine the costs of the following instruction sequences:

LD R0, y
LD R1, z
ADD R0, R0, R1
ST x, R0

$$2 + 2 + 1 + 2 = 7$$

LD R0, p
LD R1, x
ST 0(R0), R1

$$2 + 2 + 2 = 6$$

LD R0, i
MUL R0, R0, 8
LD R1, a(R0)
ST b, R1

$$2 + 2 + 2 + 2 = 8$$

LD R0, x
LD R1, y
SUB R0, R0, R1
BLTZ *R3, R0

$$2 + 2 + 1 + 1 = 6$$

- Generated code sequence for basic block:

```
t1 := a + b  
t2 := c + d  
t3 := e - t2  
t4 := t1 - t3
```

```
MOV a , R0  
ADD b , R0  
MOV c , R1  
ADD d , R1  
MOV R0 , t1  
MOV e , R0  
SUB R1 , R0  
MOV t1 , R1  
SUB R0 , R1  
MOV R1 , t4
```



```
x = a + b + c + d + e + f;  
y = a + c + e;
```

```
t1 = a + b  
t2 = t1 + c  
t3 = t2 + d  
t4 = t3 + e  
t5 = t4 + f  
x = t5  
t6 = a + c  
t7 = c + e  
y = t6 + t7
```

```
t1 = a + c  
t2 = t1 + e  
y = t2  
t3 = t2 + b  
t4 = t3 + d  
t5 = t4 + f  
x = t5
```